PROJECT REQUIREMENTS DOCUMENT

Posit-Optimised YOLOv8n on FPGA

1D Systolic PDPU Array • Posit⟨16,1⟩ • Mitchell Approximate Arithmetic • Im2col Conv Mapping

---

Target Platform: Xilinx Zynq UltraScale+ ZCU104 (XCZU7EV)
Accelerator Architecture: 1D Weight-Stationary Systolic Array of PDPU units
Number Format: Posit⟨16, es=1⟩ with 3-bit regime cap
Approximate Arithmetic: Mitchell log-based multiplier (with error correction) + TBD adder
Conv Mapping: Im2col → dot product
Document Version: 0.3 — Draft
Classification: Academic Research
Project Duration: 25 days

1. Introduction and Motivation

This document defines the requirements, scope, and technical constraints for the design and implementation of a Posit-arithmetic-optimised YOLOv8 Nano (YOLOv8n) object detection accelerator on the Xilinx Zynq UltraScale+ ZCU104 FPGA. The accelerator is built around a 1D weight-stationary systolic array of Posit Dot-Product Units (PDPUs) [5], operating in Posit⟨16, es=1⟩ arithmetic [1][4] with Mitchell's logarithm-based approximate multiplication [Mitchell 1962] and a to-be-confirmed approximate adder. Convolutions are mapped via im2col, reducing all Conv2d operations to dot products regardless of kernel shape.

Posit numbers concentrate representable precision near 1.0, where BN-folded neural network activations cluster, while gracefully extending dynamic range toward the extremes [1][3][4]. Prior work (Deep Positron [3], PDPU [5], Posit accelerator evaluation [7]) validates Posit arithmetic for deep learning. To the best of the team's knowledge, no published work applies Posit arithmetic to YOLOv8 on FPGA, making this project a novel academic contribution. The full design, verification, and benchmarking must be completed within 25 days.

---

2. Project Scope

2.1 In Scope

- YOLOv8n full inference pipeline (Backbone, Neck, Head) accelerated on ZCU104 PL fabric [2][6].
- 1D weight-stationary systolic array of PDPU units as the sole compute engine, shared across all layers by time-multiplexing.
- Tiered array scaling: 1 PE (Tier 1) → 4 PEs (Tier 2) → 8 or 16 PEs (Tier 3); final size determined by post-Tier-2 resource report.
- Posit⟨16, es=1⟩ uniform data format; 3-bit regime cap; activations remain in Posit16 between all layers with no intermediate encode/decode.
- Im2col preprocessing buffer: flattens 3×3×C_in or 1×1×C_in input patches into flat vectors before feeding the array.
- 1-hop inter-PE pipeline registers; upgrade to 2-hop during Vivado implementation if timing closure requires it.
- Weights pre-loaded from LPDDR4 into on-chip BRAM/URAM via AXI DMA before each layer computation begins.
- Mitchell's log-based approximate multiplier with first-order error correction inside each PE.
- Approximate adder (variant TBD — Open Item #2) inside each PE.
- Wide integer accumulator per PE (32–48 bits TBD); binary reduction adder tree at dot-product completion; single Posit16 rounding on output.
- Posit HDL sourced from Posit-HDL-Arithmetic and PACoGen; modifications version-controlled with upstream diffs documented.
- Accuracy benchmarked against float32 PyTorch YOLOv8n and INT8 baselines on COCO val2017 [6][8].
- Resource (LUT, DSP, BRAM, URAM, FF) and power measured via Vivado post-implementation reports [7].

2.2 Out of Scope

- 2D systolic array topology — deferred due to 25-day timeline and debugging complexity.
- In-network Posit encode/decode — handled offline (input) and on PS (output) only.
- Training in Posit arithmetic — pre-trained float32 weights converted offline.
- Full Quire exact accumulator — replaced by wide integer accumulator with single rounding per MAC output (see Section 3.4).
- YOLOv8 variants other than Nano.
- Real-time video streaming, ASIC tapeout.

---

3. System Overview

3.1 YOLOv8n Pipeline Summary

YOLOv8n follows a three-stage pipeline [2]: (1) Backbone — five-scale CSPNet pyramid (P1–P5) of C2f modules and SPPF, producing feature maps at P3 (80×80), P4 (40×40), P5 (20×20) for the Neck; (2) Neck — FPN top-down semantic path fused with PAN bottom-up localisation path across P3/P4/P5; (3) Head — decoupled anchor-free branches for box regression (CIoU + DFL) and classification (BCE). All three stages time-multiplex through the single shared systolic array on PL; the PS handles weight DMA, preprocessing, and post-processing.

3.2 End-to-End Data Flow

- Offline: float32 weights converted to Posit16; BN parameters folded into conv weights; im2col parameters configured per layer.
- Layer start: PS DMA copies Posit16 weight tensor from LPDDR4 into BRAM/URAM banks on PL via AXI4.
- Im2col buffer: hardware extracts 3×3×C_in (or 1×1×C_in) patches from input feature map BRAM and presents them as flat Posit16 vectors to the array.
- Systolic compute: weight vector pre-loaded into PE registers; activation vector shifts left-to-right through PEs at one element per clock per hop; each PE accumulates one partial product per cycle.
- Reduction: per-PE accumulators drain into binary adder tree; result rounded to Posit16 and written to output feature map BRAM.
- Layer end: output BRAM becomes input BRAM for the next layer (ping-pong where BRAM budget allows).
- Pipeline end: PS reads Posit16 detection outputs; performs NMS and display in float32.

3.3 Systolic Array Architecture

The accelerator is a 1D weight-stationary systolic array of N PDPU Processing Elements (PEs), where N is in {1, 4, 8, 16} set by the tiered scaling strategy (Section 5). Each PE contains one approximate MAC unit (Mitchell multiplier + approximate adder) and a wide integer accumulator. Operation per dot product:

- Weights stationary: each PE loads one Posit16 weight from BRAM before computation; this weight is held fixed for the duration of one output channel.
- Activations shift: the im2col activation vector enters PE[0] and propagates right through PE[1]…PE[N-1] via 1-hop registers at one element per clock.
- Each PE multiplies its fixed weight by the arriving activation and accumulates the product into its wide integer register.
- After all input elements have shifted through, partial sums drain into a $\log_2(N)$-depth binary reduction adder tree, producing one wide integer result, which is rounded to Posit16 and written to output BRAM.
- To compute all output channels: weights are reloaded per output channel; the activation vector is replayed. Intermediate feature maps persist in BRAM between layers.

Illustrative 4-PE dot product cycle:

PE[0]	PE[1]	PE[2]	PE[3]
Weight w0 (fixed)	Weight w1 (fixed)	Weight w2 (fixed)	Weight w3 (fixed)
Act: a0 →	→ a1 →	→ a2 →	→ a3 (shift, 1-hop)
Acc += w0 x a0	Acc += w1 x a1	Acc += w2 x a2	Acc += w3 x a3

PE[0]	PE[1]	PE[2]	PE[3]
Reduction adder tree (log2(N) levels) → single Posit16 rounded output			

3.4 Number Format Specification

Parameter	Value	Rationale
Format	Posit⟨16, es=1⟩	16-bit; es=1 maximises fraction bits; useed=4 [4][5]
Regime cap	3 bits (k ∈ {-3…+3})	Range [1/64, 64]; validate empirically first (Open Item #4)
Representable range	4^-3 to 4^3 = 1/64 to 64	Expected to cover all post-BN activations
Rounding	Round-to-nearest, once per MAC output	Single rounding after full accumulation drains
Accumulator width	Wide integer, 32-48 bits (TBD)	Prevents overflow during partial sum chain
Inter-PE registers	1-hop (upgrade to 2-hop if timing fails)	Simplest correct pipeline; timing fix available
Special values	0 and NaR	No NaN exceptions; NaR as Quiet NaN [1][9]

Quire Trade-off Discussion

The Quire is a 512-bit exact accumulator that eliminates all rounding within a dot product [1][5][9], as used in the original PDPU [5]. On FPGA, a 512-bit carry-propagate accumulator per PE consumes substantial LUTs and limits clock frequency. This project substitutes a wide integer accumulator (32-48 bits) per PE with a single final round-to-nearest. This is a deliberate trade-off for the 25-day timeline: some rounding error is reintroduced versus the full Quire, but hardware cost is substantially reduced. The rounding error magnitude per layer must be measured during Tier 1 simulation and compared against a Quire-exact reference to quantify the accuracy cost numerically.

---

4. Functional Requirements

4.1 Im2col Preprocessing Buffer

Im2col flattens each 3x3xC_in input patch around an output pixel location into a flat vector of length 9xC_in. The dot product of this vector with the corresponding weight vector produces one output activation value. Stacking all output-pixel vectors enables all convolutions to be expressed uniformly as matrix-vector products, regardless of kernel size.

- Hardware buffer reads from input feature map BRAM and presents one flat Posit16 vector per compute group to the array.
- 1x1 convolutions degenerate naturally: patch length = C_in; same buffer path, no reconfiguration.
- Same-padding for 3x3 kernels: zero-pad boundary positions with Posit16 zero (0x0000).
- Buffer reconfigurable per layer via control registers: kernel size (1 or 3), feature map H x W, C_in, stride (fixed at 1).

4.2 PE Requirements

- Each PE: one Mitchell approximate multiplier (with error correction) + one approximate adder + wide integer accumulator.
- Weight register: one 16-bit Posit register loaded from BRAM at layer start; stationary during one output channel computation.
- Activation input: 16-bit Posit from left neighbour via 1-hop register; forwarded to right neighbour simultaneously.
- Accumulator: wide integer (32-48 bits); cleared at dot-product start; accumulates products over 9xC_in cycles.
- Output: accumulator value passed to reduction tree at completion; PE resets for next output channel.
- Each PE independently simulatable and testable in isolation (Tier 1 mandatory requirement).

4.3 Reduction Adder Tree

- Binary tree of approximate Posit16 adders; depth = log2(N) for N PEs.
- N=4: 2-level tree, 3 adder instances. N=8: 3-level, 7 adders. N=16: 4-level, 15 adders.
- Same approximate adder module as inside the PE MAC for implementation consistency.
- Tree output: one wide integer value rounded to Posit16 and written to output BRAM.

4.4 Weight Loading and Memory Interface

- All Posit16 layer weights stored in LPDDR4 (converted offline). Before each layer, PS DMA copies required weight tensor to BRAM/URAM via AXI4.
- Weight BRAM: one 16-bit wide bank per PE, holding weights for all output channels sequentially. Weight address controller sequences BRAM reads: one weight per PE per output channel cycle.
- Feature map BRAM: separate banks for input and output feature maps; double-buffered where BRAM budget permits to overlap computation with next-layer DMA.

4.5 Layer Controller FSM

- Top-level FSM per layer: (1) weight DMA from LPDDR4 to BRAM, (2) im2col buffer configuration, (3) systolic array compute over all output channels, (4) output feature map writeback.
- Handles all YOLOv8n layer types: Conv2d (1x1 and 3x3), C2f bottleneck chains, SPPF max-pool, Upsample (nearest-neighbour), Concat, residual Add.

- Residual additions (C2f shortcuts) use the approximate adder directly on Posit16 BRAM values — no decode-add-encode.
- SPPF max-pool: exact Posit16 comparators only; no multiplication.
- Upsample: nearest-neighbour implemented as BRAM address duplication — zero arithmetic.
- Concat: channel-dimension concatenation via BRAM bank addressing — zero arithmetic.

4.6 Layer Arithmetic Summary

Layer	Location	Arithmetic	Implementation Note
Conv2d 3x3	Backbone	Approx Posit16 MAC (array)	Im2col patch → dot product; dominant compute
Conv2d 1x1	Neck, Head	Approx Posit16 MAC (array)	Degenerate im2col; same array path
C2f Bottleneck	Backbone P1-P5	Approx Posit16 MAC + adder	Conv chains + Posit16 residual add
SPPF MaxPool	End of Backbone	Exact Posit16 compare	No multiply; comparator only
Upsample	Neck FPN	None	Nearest-neighbour via BRAM addressing
Concat	Neck skip	None	Channel concat via BRAM bank merge
Residual Add	C2f shortcuts	Approx Posit16 adder	Direct on Posit16 values; no encode/decode
BN + SiLU	All Conv blocks	BN folded offline; SiLU TBD (Open Item #6)	BN eliminated; SiLU via LUT or PWL approx
Box + Cls Branches	Head	Approx Posit16 MAC (array)	Box to PS; sigmoid for Cls on PS in float32

---

5. Tiered Array Scaling and Debug Strategy

Given the 25-day timeline, the array is built and validated in three tiers. Each tier is a hard milestone gate: the next tier begins only after current tier exit criteria are fully met. This isolates integration bugs at the smallest possible scale before each expansion.

Tier	Size	What Is Validated	Exit Criteria
T1	1 PE	Mitchell mult, approx adder, accumulator, Posit16 rounding. Weight load from BRAM. Im2col buffer for one patch. Saturation policy.	PE output matches exact Posit16 reference within error budget on 1000+ test vectors. Error characterisation documented.

Tier	Size	What Is Validated	Exit Criteria
T2	4 PEs	1-hop shift registers. Weight loading to 4 PEs. 3-adder reduction tree. Full dot product on known vector. Layer controller FSM basics. Resource headroom check.	4-PE dot product matches T1 single-PE result on same inputs. Post-synthesis resource report confirms headroom for T3. Final PE count (8 or 16) decided.
T3	8 or 16 PEs	Full YOLOv8n layer sequence. Complete pipeline. ZCU104 timing closure. mAP benchmarking.	Design meets >=100 MHz. Utilisation within targets. mAP within 5% of float32 baseline on COCO val2017.

Debug Aids Per Tier

- T1: Drive PE directly from HDL testbench; compare output to SoftPosit software reference [softposit library]; verify saturation policy on out-of-range inputs.
- T2: Inject known weight and activation vectors; tap per-PE accumulator values via Vivado ILA before the reduction tree to isolate shift register and reload bugs.
- T3: ILA on array input/output buses; inject single COCO image; read bounding box outputs on PS via UART before running full mAP sweep.

---

6. Hardware and Platform Requirements

6.1 Target Platform

Parameter	Specification
FPGA Board	Xilinx Zynq UltraScale+ ZCU104
Device	XCZU7EV-2FFVC1156
CLB LUTs	~230,400
DSP48E2 Slices	1,728
Block RAM	312 x 36 Kb
UltraRAM	96 x 288 Kb
On-board Memory	4 GB LPDDR4 (weight and feature map storage)
PS-PL Interface	AXI4 for DMA; AXI-Lite for control registers
Design Tool	Xilinx Vivado 2022.x or later
HLS (optional)	Vitis HLS for im2col buffer and layer controller prototyping
Simulation	Vivado Simulator or ModelSim (units) + Python testbench (system-level)
On-chip Debug	Vivado ILA cores on array I/O; UART readback on PS

6.2 HDL Source Libraries

- Posit-HDL-Arithmetic (github.com/manish-kj/Posit-HDL-Arithmetic): exact Posit16 arithmetic cores; used as exact reference baseline and structural starting point for approximate variants. PDPU paper [5] informs PE structure.
- PACoGen (github.com/manish-kj/PACoGen): parametric posit generator; used to instantiate Posit<16, es=1> with regime cap applied.
- All upstream modifications version-controlled; diffs from upstream committed and documented before integration into the array.

---

7. Performance and Accuracy Requirements

7.1 Accuracy Targets

Metric	Target	Measurement Method
mAP@0.5 vs float32 baseline	<=5% degradation	COCO val2017, min. 500 images [6]
mAP@0.5:0.95 vs float32	<=10% degradation	COCO val2017
Mitchell MRE with error correction	<1% (target)	Uniform random Posit16 pair sweep
Rounding error vs Quire (per layer)	Measured and documented	T1 simulation: wide-acc vs SoftPosit Quire ref
Per-layer activation error vs exact Posit16	Documented per stage	Sim comparison to SoftPosit reference
Regime saturation events	<0.1% of activations (target)	PyTorch activation histogram; verify before HDL

Note: No prior FPGA Posit YOLOv8 implementation exists in the published literature [7]. Comparisons are made against: (1) float32 PyTorch YOLOv8n software baseline, (2) exact Posit16 simulation, (3) approximate Posit16 FPGA result (this work), (4) INT8 quantised baseline. This four-way table is the primary hardware contribution of the paper.

7.2 Hardware Performance Targets

Metric	Target	Notes
Clock frequency	>=100 MHz (200 MHz aspirational)	Upgrade to 2-hop inter-PE registers if 1-hop fails timing
LUT utilisation	<70% of 230,400	Routing margin; ILA debug cores need headroom
DSP utilisation	Minimise	Mitchell mult targets LUT-based impl, not DSP48E2
BRAM + URAM	<80% combined	Weight banks + feature map double-buffers

Metric	Target	Notes
Area vs exact Posit16 array	>=5% LUT reduction (aspirational)	From approximate arithmetic units
Power vs float32 FPGA baseline	>=5% reduction (aspirational)	Vivado post-implementation power analysis

---

8. Approximate Computing Strategy

8.1 Mitchell's Logarithm-Based Multiplier with Error Correction

Mitchell's algorithm (1962) approximates multiplication by mapping operands to base-2 logarithms, performing integer addition, and mapping back. For Posit<16, es=1>, the regime and exponent fields directly encode the coarse power-of-two scale (useed^k x 2^e), which maps directly onto the integer part of the log without additional leading-zero detection logic. This is a key implementation advantage over IEEE 754.

- Step 1 - Log encoding: extract sign s, regime k, exponent e, fraction f from each Posit16 operand. Log-domain integer = (2k + e); fractional part = f.
- Step 2 - Log-domain addition: L_result = L_A + L_B.
- Step 3 - Error correction: subtract f_A x f_B from L_result. Reduces MRE from ~3% (uncorrected) to below 1%.
- Step 4 - Antilog: map corrected L_result back to Posit16 (integer part to regime+exponent; fractional part to fraction field).
- Step 5 - Sign: XOR input signs; handle 0 and NaR as special cases.

The correction term f_A x f_B multiplies two sub-word fractional parts in [0,1). This can be implemented as a small LUT or shift-and-add unit (Open Item #7) without recursing into another full Mitchell multiplier.

8.2 Approximate Adder (TBD)

Variant under selection. Once finalised, the following will be added here: variant name and reference, error characterisation (MRE, error distance histogram), area and Fmax on ZCU104, and rationale for selection. Requirements for any variant:

- Must operate on Posit16 significands (aligned sign-magnitude) or directly on Posit16 encoded values.
- Must not cause sign flips or saturation events within the 3-bit regime range on typical activation values.
- Same module instantiated in PE MAC units and in the reduction adder tree for consistency.

8.3 No Encode/Decode at Layer Boundaries

All internal data paths carry raw Posit<16,1> bit-vectors. The only permissible format conversions are: (1) network input - float32 to Posit16 offline; (2) network output - Posit16 to float32 on PS for

NMS. Im2col buffer outputs, BRAM transfers, residual additions, and reduction tree inputs/outputs all carry raw Posit16 without intermediate decode.

---

9. Verification and Validation Plan

9.1 Tier 1 - Single PE Unit Tests

- Mitchell multiplier: 10,000+ Posit16 input pair sweep; MRE distribution plot; confirm correction reduces MRE below 1%; compare to exact Posit16 multiplier.
- Approximate adder: same sweep methodology once variant is confirmed.
- Full PE MAC: verify on known dot-product vectors; compare wide-accumulator result against SoftPosit Quire reference to quantify rounding error cost.
- Saturation test: inject activations exceeding regime cap; confirm saturation policy triggers without silent propagation.

9.2 Tier 2 - 4-PE Array Integration Tests

- Inject known 4-element weight and activation vectors; verify per-PE accumulators via ILA before reduction tree.
- Verify 1-hop shift timing: activation at PE[k] on cycle k, PE[k+1] on cycle k+1.
- Verify weight reload between output channels does not corrupt in-progress computation.
- Verify reduction tree output matches expected dot product value.

9.3 Tier 3 - Full System Validation

- Full YOLOv8n inference on representative COCO images; verify detection outputs via UART on PS before mAP sweep.
- mAP on COCO val2017 (minimum 500 images; 5,000 if time permits) [6].
- Four-way comparison table: float32 PyTorch / exact Posit16 / approx Posit16 FPGA (this work) / INT8 quantised.
- Resource and power from Vivado post-implementation report; tabulated against baselines.

---

10. Open Items and Pending Decisions

#	Open Item	Owner	Target Resolution
1	Regime cap saturation policy: clamp to NaR or to max representable Posit16 value?	Team	Before T1 HDL coding
2	Approximate adder: finalise variant; add name, reference, and error characterisation to Section 8.2	Team	Before PE MAC design
3	Accumulator width: 32-bit or 48-bit integer per PE?	Team	Before PE MAC design

#	Open Item	Owner	Target Resolution
4	Validate 3-bit regime cap via PyTorch activation histogram BEFORE any HDL coding	Team	Days 1-2 (priority)
5	Weight conversion tool: Python or C++?	Team	Before system integration
6	SiLU activation implementation: Posit16 LUT or piecewise-linear approximation?	Team	After T1 PE validated
7	Mitchell correction term (fA x fB): small LUT or shift-and-add?	Team	During T1 multiplier design
8	Final PE count (8 or 16): decided after T2 post-synthesis resource report	Team	End of Tier 2
9	COCO evaluation subset size given 25-day compute budget	Team	Tier 3 planning

---

11. 25-Day Project Timeline

Critical path: arithmetic units → Tier 1 PE → Tier 2 array → full pipeline → timing closure → benchmarking. M0 and M1 run in parallel (software team validates regime cap while HDL team characterises arithmetic units). The highest-risk window is Days 9-18 (backbone and full pipeline integration), where most inter-module bugs will surface. Buffer time is embedded in the M4 and M5 day ranges.

Days	Milestone	Deliverable / Exit Criteria	Risk Level
1-2	M0: Regime cap validation	PyTorch histogram confirms <0.1% activations exceed [1/64, 64]. Open Items #1 and #4 resolved.	Low (software only)
1-3	M1: Arithmetic units characterised	Mitchell mult + approx adder unit sims pass error budget. Open Items #2, #3, #7 resolved.	Medium (adder TBD)
3-6	M2: Tier 1 - Single PE validated	T1 exit criteria met (Section 9.1). Weight load from BRAM working. Im2col buffer correct.	Medium
6-9	M3: Tier 2 - 4-PE array validated	T2 exit criteria met (Section 9.2). Resource headroom confirmed. Open Item #8 resolved.	Medium
9-14	M4: Full backbone in Posit16	P3/P4/P5 feature maps within tolerance of PyTorch reference on 10+ COCO images.	High (most integration complexity)
14-18	M5: Full pipeline (Backbone + Neck + Head)	Detection outputs on sample images verified on PS. Layer controller sequences all layer types.	High

Days	Milestone	Deliverable / Exit Criteria	Risk Level
18-21	M6: ZCU104 timing closure	Design meets >=100 MHz. Utilisation within targets. 2-hop upgrade applied if required.	Medium (timing risk)
21-24	M7: mAP benchmarking	Four-way COCO val2017 mAP comparison table produced.	Low (if pipeline works)
24-25	M8: Results write-up	Resource, power, and accuracy tables complete. Hardware results section drafted.	Low

---

12. References

[1] J. L. Gustafson and I. T. Yonemoto. *"Beating Floating Point at its Own Game: Posit Arithmetic."* Supercomputing Frontiers and Innovations, vol. 4, no. 2, 2017. https://posithub.org/docs/Posits4.pdf

[2] M. Yaseen. *"What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector."* arXiv preprint arXiv:2408.15857, 2024. https://arxiv.org/abs/2408.15857

[3] S. Carmichael, H. F. Langroudi, C. Khazanov, J. Lillie, J. L. Gustafson, and D. Kudithipudi. *"Deep Positron: A Deep Neural Network Using the Posit Number System."* Design, Automation & Test in Europe Conference (DATE), 2019. https://arxiv.org/pdf/1812.01762

[4] I. Chisti and A. Hossain. *"The Accuracy and Efficiency of Posit Arithmetic."* arXiv preprint arXiv:2109.08225, 2021. https://arxiv.org/pdf/2109.08225

[5] G. Murillo, A. Del Barrio, and G. Botella. *"PDPU: An Open-Source Posit Dot-Product Unit for Deep Learning Applications."* arXiv preprint arXiv:2302.01876, 2023. https://arxiv.org/pdf/2302.01876

[6] Y. Bochkovskiy, M. Jocher, et al.. *"Real-Time Flying Object Detection with YOLOv8."* arXiv preprint arXiv:2305.09972, 2023. https://arxiv.org/pdf/2305.09972

[7] M. Murillo et al.. *"Evaluation of POSIT Arithmetic with Accelerators."* arXiv preprint arXiv:2401.14117, 2024. https://arxiv.org/pdf/2401.14117

[8] X. Zhang et al.. *"YOLO V8 Network Lightweight Method Based on Pruning and Quantization."* Semantic Scholar preprint, 2024. https://pdfs.semanticscholar.org/f8be/d950392d97e52e5e31416d4a26e25d6e9ce5.pdf

[9] J. L. Gustafson. *"Posit Arithmetic."* Posit Hub Technical Report, 2017. https://posithub.org/docs/Posits4.pdf

HDL Libraries and Other Sources:

Posit-HDL-Arithmetic: https://github.com/manish-kj/Posit-HDL-Arithmetic
PACoGen: https://github.com/manish-kj/PACoGen
J. N. Mitchell, "Computer Multiplication and Division Using Binary Logarithms," IRE Trans. on Electronic Computers, vol. EC-11, no. 4, pp. 512-517, 1962.
Xilinx ZCU104 Evaluation Board User Guide (UG1267). Xilinx/AMD, 2022.